

Documentation Complète - SmartMaintain V7

Structure et Rôle de Chaque Fichier et Dossier

RACINE DU PROJET

Fichiers de Configuration

`.dockerignore`

Type: Configuration Docker

Rôle: Liste les fichiers/dossiers à exclure lors de la construction des images Docker

Contenu typique:

- `node_modules/` (évite de copier les dépendances npm)
- `.git/` (historique git non nécessaire)
- `*.md` (documentation non nécessaire en production)

`.env`

Type: Configuration Environnement

Rôle: Variables d'environnement sensibles (mots de passe, clés API)

⚠ **IMPORTANT:** Ne JAMAIS commit ce fichier (contient secrets)

Variables clés:

- `POSTGRES_PASSWORD` : Mot de passe base de données
- `JWT_SECRET` : Clé secrète pour tokens d'authentification
- `ML_MODEL_PATH` : Chemin vers modèles ML
- `REDIS_URL` : URL de connexion Redis

`.env.example`

Type: Template Configuration

Rôle: Exemple de fichier `.env` sans valeurs sensibles

Usage: Les développeurs copient ce fichier vers `.env` et remplissent les valeurs

.gitignore

Type: Configuration Git

Rôle: Liste les fichiers/dossiers à ignorer par Git

Exclut:

- `.env` (secrets)
- `node_modules/` (dépendances)
- `__pycache__/` (fichiers compilés Python)
- `.venv*/` (environnements virtuels)

docker-compose.yml

Type: Orchestration Docker

Rôle: Définit et orchestre tous les services de l'application

Services définis:

1. **postgres**: Base de données PostgreSQL 15
2. **redis**: Cache et message broker
3. **mosquitto**: Broker MQTT pour IoT
4. **migrations**: Initialisation schéma DB
5. **auth**: Microservice authentification
6. **iot**: Microservice gestion capteurs IoT
7. **ml**: Microservice Machine Learning
8. **alertes**: Microservice gestion alertes
9. **gateway**: API Gateway (point d'entrée unique)
10. **frontend**: Application React (interface utilisateur)

mosquitto.conf

Type: Configuration MQTT

Rôle: Configuration du broker MQTT Eclipse Mosquitto

Paramètres:

- Port d'écoute: 1883
- Autorisation: `allow_anonymous true` (dev)
- Persistence: Sauvegarde messages

RUN.bat

Type: Script Windows

Rôle: Script de démarrage rapide pour Windows

Commandes: `docker compose up -d`

Fichiers Documentation

README.md

Type: Documentation Principale

Rôle: Point d'entrée de la documentation du projet

Sections:

- Présentation du projet
- Fonctionnalités principales
- Technologies utilisées
- Installation et démarrage
- Architecture générale

START_HERE.md

Type: Guide Démarrage

Rôle: Guide pas-à-pas pour démarrer l'application

Contenu:

- Prérequis (Docker, Node.js)
- Commandes de démarrage
- Identifiants de test
- URLs d'accès
- Troubleshooting

DOSSIER /backend

Structure Générale

```
backend/
├── auth/           → Authentification et gestion utilisateurs
├── alertes/        → Gestion des alertes et notifications
├── iot/            → Collecte et traitement données capteurs
├── ml/             → Modèles ML et prédictions
├── gateway/        → API Gateway (reverse proxy)
├── migrations/     → Migrations base de données
├── shared/         → Code partagé entre services
└── tests/          → Tests unitaires et intégration
```

Rôle Global

Microservice responsable de l'authentification, autorisation et gestion des utilisateurs.

Structure

```
auth/
├── app.py           → Point d'entrée Flask
├── Dockerfile       → Image Docker du service
├── requirements.txt → Dépendances Python
├── models/
│   ├── __init__.py
│   └── models.py    → Modèles SQLAlchemy (User, Plant)
├── routes/
│   ├── __init__.py
│   ├── auth.py      → Routes /login, /register, /logout
│   ├── users.py      → Routes CRUD utilisateurs
│   └── plants.py     → Routes gestion usines
└── services/
    ├── __init__.py
    ├── auth_service.py → Logique métier auth
    └── user_service.py → Logique métier users
```

Fichiers Clés

app.py

- Initialise Flask
- Configure CORS
- Enregistre les blueprints (routes)
- Connecte à PostgreSQL

models/models.py

Classes:

- **User** : Utilisateurs (admin, technicien, opérateur)
- **Plant** : Usines/sites industriels
- **PlantRegistration** : Demandes d'inscription usines

routes/auth.py

Endpoints:

- `POST /login` : Authentification → JWT token
- `POST /register` : Création compte
- `POST /logout` : Déconnexion
- `GET /me` : Info utilisateur connecté

`services/auth_service.py`

Fonctions:

- `hash_password()` : Hachage bcrypt
- `verify_password()` : Vérification mot de passe
- `generate_jwt()` : Création token JWT
- `verify_jwt()` : Validation token

`/backend/alertes`

Rôle Global

Gère les alertes générées par le ML, leur cycle de vie et notifications.

Structure

```
alertes/
├─ app.py                → Point d'entrée Flask + WebSocket
├─ Dockerfile
├─ requirements.txt
├─ queries.py            → Requêtes SQL optimisées
├─ models/
│   └─ models.py        → Modèle Alert
├─ routes/
│   └─ alerts.py         → CRUD alertes
│   └─ dashboard.py     → Stats dashboard
└─ services/
    └─ alert_service.py  → Logique métier
    └─ redis_consumer.py → Consommateur Redis
    └─ exceptions.py     → Exceptions custom
```

Fichiers Clés

`app.py`

- Flask + Flask-SocketIO (WebSocket)

- Écoute Redis pour nouvelles alertes
- Broadcast temps réel aux clients

`models/models.py`

Classe Alert:

- `machine` : Équipement concerné
- `defect` : Type de défaut détecté
- `severity` : Critique/Majeure/Mineure
- `status` : open/assigned/acknowledged/resolved
- `assigned_to` : Technicien assigné
- `anomaly_score` : Score d'anomalie (0-1)

`routes/alerts.py`

Endpoints:

- `GET /api/alertes` : Liste alertes (filtrable)
- `GET /api/alertes/:id` : Détail alerte
- `PATCH /api/alertes/:id` : Modifier alerte
 - Actions: assign, acknowledge, resolve, reopen

`services/alert_service.py`

Fonctions:

- `assign_alert()` : Assigner à technicien
- `acknowledge_alert()` : Acquitter alerte
- `resolve_alert()` : Résoudre/clôturer
- `reopen_alert()` : Réouvrir alerte

`services/redis_consumer.py`

- Thread consommateur Redis pub/sub
 - Écoute canal `defects:new`
 - Crée alertes en base de données
 - Broadcast WebSocket
-

Rôle Global

Collecte données des capteurs IoT, replay simulation, transmission au ML.

Structure

```
iot/
├── app.py                → Point d'entrée Flask
├── Dockerfile
├── requirements.txt
├── queries.py            → Requêtes SQL capteurs
├── models/
│   └── models.py        → Modèles Sensor, SensorReading
├── routes/
│   └── iot.py           → Routes capteurs
└── services/
    ├── config_service.py → Config capteurs
    ├── replay_service_v7.py → Simulation replay
    └── __init__.py
```

Fichiers Clés

app.py

- Connecte MQTT broker (Mosquitto)
- Reçoit messages capteurs
- Stocke lectures en base
- Transmet au ML via Redis

models/models.py

Classes:

- **Sensor** : Configuration capteur (nom, type, unité)
- **SensorReading** : Lecture capteur (valeur, timestamp)

services/replay_service_v7.py

Fonction: Simulation données capteurs

- Charge fichiers .npz (datasets)
- Replay données historiques
- Publie sur MQTT à intervalle régulier

- Usage: Mode développement sans capteurs physiques

/backend/ml

Rôle Global

Modèles CNN pour détection défauts, extraction features, prédictions.

Structure

```
ml/
├─ app.py                → Point d'entrée Flask
├─ Dockerfile
├─ requirements.txt
├─ models/               → Fichiers .keras (modèles entraînés)
│   ├── MOTEUR_model.keras
│   ├── COMPRESSEUR_model.keras
│   └─ ...
├─ routes/
│   ├── predict.py       → Endpoint prédiction
│   └─ status.py         → Health check
├─ engines/
│   ├── unified_engine.py → Moteur unifié ML
│   ├── feature_extractor_v7.py → Extraction features
│   └─ __init__.py
└─ services/
    ├── ml_service.py    → Orchestration ML
    └─ model_service_v7.py → Chargement modèles
```

Fichiers Clés

app.py

- Thread consommateur Redis
- Écoute `sensor_window_ready`
- Déclenche prédictions
- Publie résultats sur `defects:new`

routes/predict.py

Endpoint: `POST /api/ml/predict`

- Reçoit fenêtre de données capteurs

- Extrait features (FFT, stats, entropie)
- Prédit avec CNN
- Retourne défaut + score confiance

`engines/unified_engine.py`

Classe UnifiedEngine:

- Charge modèles Keras
- Gère prédictions multi-machines
- Cache modèles en mémoire

`engines/feature_extractor_v7.py`

Extraction Features:

- **Temporelles:** moyenne, std, min, max, RMS
- **Fréquentielles:** FFT, spectres, harmoniques
- **Statistiques:** skewness, kurtosis, entropie
- **Fenêtrage:** 10 secondes de données

`models/` (dossier)

Contenu: Fichiers `.keras` (modèles CNN entraînés)

- MOTEUR_model.keras: Détection défauts moteurs
- COMPRESSEUR_model.keras: Défauts compresseurs
- POMPE_model.keras: Défauts pompes
- etc.

Format: TensorFlow/Keras SavedModel

 `/backend/gateway`

Rôle Global

Point d'entrée unique (API Gateway), routing, CORS, WebSocket.

Structure

```
gateway/
├─ app.py           → Point d'entrée Flask
├─ Dockerfile
├─ requirements.txt
└─ routes/
    └─ __init__.py
```

Fichiers Clés

app.py

Responsabilités:

1. **Reverse Proxy**: Redirige requêtes vers microservices
 - `/api/auth/*` → service auth
 - `/api/alertes/*` → service alertes
 - `/api/iot/*` → service iot
 - `/api/ml/*` → service ml
2. **CORS**: Configure Cross-Origin Resource Sharing
 - Autorise frontend localhost:3000
3. **WebSocket Aggregation**:
 - Connecte aux WebSocket services alertes
 - Broadcast agrégé aux clients frontend
4. **Health Checks**: Vérifie santé des services

 **/backend/migrations**

Rôle Global

Initialisation et migrations schéma base de données.

Structure

```
migrations/
├─ app.py           → Script migration
├─ Dockerfile
├─ requirements.txt
└─ init_schema.sql  → Schéma SQL initial (optionnel)
```

Fichiers Clés

`app.py`

Exécution:

1. Attend que PostgreSQL soit prêt
2. Crée tables via SQLAlchemy (si absentes)
3. Insère données de seed (admin, démo)
4. S'arrête (restart: no dans docker-compose)

Tables créées:

- `users` : Utilisateurs
- `plants` : Usines
- `plant_registrations` : Inscriptions
- `sensors` : Configuration capteurs
- `sensor_readings` : Lectures capteurs
- `alerts` : Alertes

 `/backend/shared`

Rôle Global

Code partagé entre tous les microservices (utilitaires, constantes).

Structure

```
shared/
├── __init__.py
├── auth.py           → Décorateurs authentification
├── config.py         → Configuration centralisée
├── constants.py      → Constantes globales
├── http.py           → Helpers HTTP (json_response, etc.)
└── ml_config.py      → Config modèles ML
```

Fichiers Clés

`auth.py`

Décorateurs:

- `@require_auth()` : Vérifie JWT token

- `@require_role(['admin'])` : Vérifie rôle utilisateur
- `get_current_user()` : Extrait user du token

`constants.py`

Constantes:

- `ROLES` : admin, superviseur, technicien, operateur
- `ALERT_STATUSES` : open, assigned, acknowledged, resolved
- `SEVERITIES` : Critique, Majeure, Mineure
- `MACHINES` : MOTEUR-001, COMPRESSEUR-001, etc.

`ml_config.py`

Configuration ML:

- `WINDOW_SIZE` : 10 secondes
- `SAMPLING_RATE` : 48000 Hz
- `N_FFT` : 2048
- `DEFECT_THRESHOLD` : 0.7
- Mapping machines → modèles

`/backend/tests`

Rôle Global

Tests unitaires et intégration des services backend.

Structure

```
tests/
├─ __init__.py
├─ conftest.py           → Fixtures pytest
├─ test_alert_service.py → Tests alertes
├─ test_auth_service.py  → Tests auth
├─ test_iot_window_buffers.py → Tests IoT
├─ test_ml_predict_api.py → Tests ML
├─ test_model_service.py → Tests modèles
└─ test_plant_id_propagation.py → Tests multi-tenant
```

Fichiers Clés

`conftest.py`

Fixtures pytest:

- `app` : Instance Flask test
- `client` : Client HTTP test
- `db` : Base de données test

`test_*.py`

Structure type:

```
def test_login_success(client):  
    response = client.post('/api/auth/login', json={...})  
    assert response.status_code == 200  
    assert 'access_token' in response.json
```

DOSSIER `/frontend`

Structure Générale

```
frontend/  
├─ src/           → Code source React  
├─ public/        → Assets statiques  
├─ dist/          → Build de production (généré)  
├─ node_modules/  → Dépendances npm (généré)  
├─ package.json   → Manifeste npm  
├─ vite.config.js  → Config Vite (bundler)  
├─ tailwind.config.js → Config Tailwind CSS  
├─ Dockerfile      → Image Docker Nginx  
└─ nginx.conf      → Config serveur web
```

/frontend/src

Structure

src/	
├─ main.jsx	→ Point d'entrée React
├─ App.jsx	→ Composant racine + routing
├─ index.css	→ Styles globaux
├─ components/	→ Composants réutilisables
├─ pages/	→ Pages de l'application
├─ services/	→ Clients API
├─ hooks/	→ Hooks React custom
├─ utils/	→ Fonctions utilitaires
└─ constants/	→ Constantes frontend

/frontend/src/components

Rôle

Composants UI réutilisables partagés.

Fichiers

Layout.jsx

Rôle: Layout principal avec sidebar, header, navigation

Contenu: Menu, logout, profil utilisateur

RequireAuth.jsx

Rôle: HOC (Higher-Order Component) pour protection routes

Usage: Vérifie token JWT, redirige vers /login si absent

LoadingSpinner.jsx

Rôle: Indicateur de chargement animé

Props: `size` (sm/md/lg), `fullScreen` (boolean)

EmptyState.jsx

Rôle: État vide avec icône et message

Props: `icon`, `title`, `description`

ErrorBoundary.jsx

Rôle: Capteur d'erreurs React (error boundary)

Usage: Entoure l'application pour catch erreurs

SkeletonLoader.jsx

Rôle: Placeholders animés pendant chargement

Variants: KPISkeleton, MachineCardSkeleton, AlertListSkeleton

Icons.jsx

Rôle: Bibliothèque 14 icônes SVG custom

Icônes: AlertIcon, ChartIcon, UserIcon, ClockIcon, etc.

/frontend/src/pages

Rôle

Pages complètes de l'application (routes).

Fichiers

LoginPage.jsx

Route: **/login**

Rôle: Authentification utilisateur

Features: Formulaire login, gestion erreurs

DashboardPage.jsx

Route: **/dashboard**

Rôle: Vue d'ensemble KPIs

Features:

- KPIs (machines actives, alertes, disponibilité)
- Liste machines avec statuts
- Alertes récentes
- Export PDF
- Filtres temporels (1h/24h/7j/30j)
- Drill-down cliquable vers Surveillance

SurveillancePage.jsx

Route: **/surveillance**

Rôle: Monitoring temps réel capteurs

Features:

- Onglets par machine
- Graphiques Area charts (valeurs capteurs)
- Score d'anomalie (gauge)
- Probabilités défauts
- Heatmap 24h alertes
- Historique défauts

AlertesPage.jsx

Route: `/alertes`

Rôle: Gestion alertes

Features:

- Vue tableau uniquement
- Filtres (machine, sévérité, statut, date)
- Actions en lot (multi-sélection)
- Assignment techniciens
- Acquittement / Résolution
- Toast notifications

AlertDetailPage.jsx

Route: `/alertes/:id`

Rôle: Détail d'une alerte

Features:

- Modal assignation technicien
- 4 boutons actions (Assigner/Acquitter/Résoudre/Réouvrir)
- Timeline visuelle statuts
- Informations complètes

ComposantsPage.jsx

Route: `/composants`

Rôle: Gestion équipements (admin)

Features: CRUD machines, capteurs associés

UtilisateursPage.jsx

Route: `/utilisateurs`

Rôle: Gestion utilisateurs (admin)

Features: CRUD users, attribution rôles

ProfilePage.jsx

Route: /profil

Rôle: Profil utilisateur connecté

Features: Édition infos perso, changement mot de passe

PlantProfilePage.jsx

Route: /usine/profil

Rôle: Profil usine (admin)

Features: Édition infos usine

PlantRegistrationPage.jsx

Route: /inscription-usine

Rôle: Formulaire inscription nouvelle usine

Accès: Public (pas de login requis)

SuperAdminPlantsPage.jsx

Route: /superadmin/usines

Rôle: Gestion usines (superadmin)

Features: Validation/rejet inscriptions

SuperAdminRegistrationsPage.jsx

Route: /superadmin/inscriptions

Rôle: Demandes inscriptions en attente

Features: Approbation usines

/frontend/src/services

Rôle

Clients API pour communication avec backend.

Fichiers

api.js

Rôle: Instance Axios configurée

Config:

- Base URL: `http://localhost:5000/api`
- Interceptors: Ajout auto JWT token
- Gestion erreurs 401 (redirect login)

authService.js

Fonctions:

- `login(email, password)` : POST /auth/login
- `logout()` : Nettoie token localStorage
- `getUser()` : Décode JWT token
- `isAuthenticated()` : Vérifie si logged in

alerteService.js

Fonctions:

- `getAlertes(filters)` : GET /alertes
- `getAlerteById(id)` : GET /alertes/:id
- `assignAlert(id, userId)` : PATCH assign
- `acknowledgeAlert(id)` : PATCH acknowledge
- `resolveAlert(id)` : PATCH resolve
- `reopenAlert(id)` : PATCH reopen

dashboardService.js

Fonctions:

- `getDashboardData()` : Agrégation KPIs
- `getMachines()` : Liste machines
- `getRecentAlerts()` : Alertes récentes

componentService.js

Fonctions:

- `getComponents()` : Liste machines
- `createComponent()` : Créer machine
- `updateComponent()` : Modifier machine
- `deleteComponent()` : Supprimer machine

userService.js

Fonctions:

- `getUsers()` : Liste utilisateurs
- `createUser()` : Créer utilisateur
- `updateUser()` : Modifier utilisateur
- `deleteUser()` : Supprimer utilisateur

socketService.js

Rôle: Connexion WebSocket Socket.io

Events:

- `alert:new` : Nouvelle alerte reçue
 - `alert:updated` : Alerte modifiée
 - `sensor:update` : Mise à jour capteur temps réel
-

/frontend/src/hooks

Rôle

Hooks React custom pour logique réutilisable.

Fichiers

useDashboard.js

Hook: `useDashboard()`

Retourne:

- `kpis` : { machinesActives, alertesCritiques, disponibilite }
- `machines` : Liste machines avec statuts
- `alerts` : Alertes récentes
- `loading` : Boolean chargement

useSurveillance.js

Hook: `useSurveillance()`

Retourne:

- `tabs` : Onglets machines disponibles
- `activeMachine` : Machine sélectionnée
- `chartSeriesBySensor` : Données graphiques capteurs
- `anomalyScore` : Score anomalie (0-100)
- `defectScores` : Probabilités par défaut
- `defectHistory` : Historique défauts

Gestion:

- Connexion WebSocket
- Mise à jour temps réel
- Cache données

/frontend/src/utils

Rôle

Fonctions utilitaires.

Fichiers

exportPDF.js

Fonctions:

- `exportDashboardToPDF()` : Génère PDF structuré
 - Utilise jsPDF
 - Contenu: KPIs, machines, alertes
- `exportDashboardScreenshot()` : Capture screenshot
 - Utilise html2canvas
 - Capture visuelle dashboard

/frontend/src/constants

Rôle

Constantes frontend.

Fichiers

machines.js

Export: `MACHINE_OPTIONS`

Contenu: Liste machines avec labels

```
[
  { value: 'MOTEUR-001', label: 'Moteur Électrique 001' },
  { value: 'COMPRESSEUR-001', label: 'Compresseur 001' },
  ...
]
```

Fichiers Configuration Frontend

package.json

Rôle: Manifeste npm

Dépendances clés:

- `react` : ^18.3.1
- `react-router-dom` : ^6.x (routing)
- `recharts` : ^2.x (graphiques)
- `tailwindcss` : ^3.x (CSS utility)
- `axios` : ^1.x (HTTP client)
- `socket.io-client` : ^4.x (WebSocket)
- `jspdf` : ^2.5.2 (export PDF)
- `html2canvas` : ^1.4.1 (screenshots)
- `react-hot-toast` : ^2.x (notifications)

Scripts:

- `npm run dev` : Serveur développement (Vite)
- `npm run build` : Build production
- `npm run preview` : Preview build

vite.config.js

Rôle: Configuration Vite (bundler)

Config:

- Plugins: React, TailwindCSS
- Optimizations: Code splitting, minification
- Dev server: Port 5173 (dev), HMR

tailwind.config.js

Rôle: Configuration Tailwind CSS

Personnalisation:

- Couleurs custom
- Breakpoints responsive
- Animations custom

Dockerfile

Rôle: Image Docker frontend

Étapes:

1. **Build stage:** `npm run build` → génère `/dist`
2. **Production stage:** Nginx Alpine
3. Copie `/dist` vers `/usr/share/nginx/html`
4. Expose port 3000

nginx.conf

Rôle: Configuration serveur Nginx

Config:

- Port 3000
- SPA fallback (index.html pour toutes routes)
- Gzip compression
- Cache headers

DOSSIER `/docs`

Fichiers

ARCHITECTURE.md

Contenu: Architecture système complète

Sections:

- Diagramme architecture microservices
- Flow données (IoT → ML → Alertes)
- Technologies stack
- Patterns utilisés

README.md

Contenu: Introduction documentation

Liens: Vers autres docs

Contenu: Cas d'usage métier

Exemples:

- Détection défaut moteur
- Assignation alerte technicien
- Export rapport maintenance



FLUX DE DONNÉES

1. Collecte Données IoT

Capteur Physique

- MQTT (Mosquitto)
- Service IoT
- PostgreSQL (sensor_readings)
- Redis (buffer fenêtre 10s)

2. Détection Défauts ML

Redis (fenêtre prête)

- Service ML
- Extraction features (FFT, stats)
- CNN Prediction
- Redis (defects:new) si défaut

3. Création Alerte

Redis (defects:new)

- Service Alertes
- PostgreSQL (alerts table)
- WebSocket broadcast
- Frontend notification temps réel

4. Gestion Alerte

Frontend (action utilisateur)

- API Gateway
- Service Alertes (assign/ack/resolve)
- PostgreSQL (update)
- WebSocket broadcast
- Frontend mise à jour

SÉCURITÉ

Authentification

- JWT tokens: Expiration 24h
- bcrypt: Hachage mots de passe (salt rounds: 12)
- CORS: Whitelist origines autorisées

Autorisation

- RBAC (Role-Based Access Control)
- Rôles:
 - `superadmin` : Gestion multi-tenant
 - `admin` : Gestion usine complète
 - `superviseur` : Vue + assignation alertes
 - `technicien` : Gestion alertes assignées
 - `opérateur` : Vue lecture seule

Multi-Tenant

- Isolation par `plant_id`
 - Requêtes filtrées automatiquement
 - Admins voient seulement leur usine
-



DÉPLOIEMENT

Développement

```
docker compose up -d
```

- Frontend: <http://localhost:3000>
- Backend: <http://localhost:5000>

Production

1. Build images:

```
docker compose build
```

2. Push registry:

```
docker tag smartmaintain-frontend registry.example.com/frontend:v7  
docker push registry.example.com/frontend:v7
```

3. Deploy:

- Kubernetes manifests
- Helm charts
- Docker Swarm

Variables Environnement Production

- `NODE_ENV=production`
 - `POSTGRES_PASSWORD=<strong-password>`
 - `JWT_SECRET=<random-256-bits>`
 - `REDIS_URL=redis://redis-prod:6379`
-



MONITORING

Logs

```
# Tous les services
docker compose logs -f

# Service spécifique
docker compose logs -f ml
```

Métriques

- PostgreSQL: Connexions, requêtes lentes
- Redis: Mémoire, hit rate
- Nginx: Requêtes/s, codes status
- Python: CPU, mémoire par service

Health Checks

- `/health` sur chaque service
- Docker healthchecks
- Readiness/liveness probes



TESTS

Backend

```
cd backend/tests
pytest -v
```

Couverture:

- Tests unitaires: Services, modèles
- Tests intégration: API endpoints
- Tests e2e: Flux complets

Frontend

```
cd frontend  
npm run test # (si configuré)
```



PERFORMANCES

Backend

- PostgreSQL: Index sur colonnes fréquentes
- Redis: Cache requêtes fréquentes
- ML: Batch predictions, modèles chargés en RAM

Frontend

- Code splitting: Lazy loading pages
 - Bundle size:
 - Dashboard: 389 KB (gzipped: 126 KB)
 - Surveillance: 99 KB (gzipped: 26 KB)
 - Optimizations: Minification, tree shaking
-



DÉVELOPPEMENT

Prérequis

- Docker Desktop 20+
- Node.js 20+
- Python 3.11+
- Git

Setup Local

```
# 1. Clone
git clone <repo>
cd smartmaintain

# 2. Config
cp .env.example .env
# Éditer .env

# 3. Start
docker compose up -d

# 4. Frontend dev (HMR)
cd frontend
npm install
npm run dev # Port 5173
```

Workflow Git

```
# Feature branch
git checkout -b feature/nouvelle-fonctionnalite

# Commit
git add .
git commit -m "feat: ajout export Excel"

# Push
git push origin feature/nouvelle-fonctionnalite

# Pull Request → Review → Merge
```



RESSOURCES

Technologies

- React: <https://react.dev>
- Flask: <https://flask.palletsprojects.com>
- TensorFlow: <https://tensorflow.org>
- Docker: <https://docs.docker.com>

- PostgreSQL: <https://postgresql.org/docs>

Documentation Interne

- `README.md` : Vue d'ensemble
 - `START_HERE.md` : Guide démarrage
 - `docs/ARCHITECTURE.md` : Architecture détaillée
 - `docs/USE_CASES.md` : Cas d'usage
-



CHECKLIST AVANT COMMIT

- ☐ Code testé localement
 - ☐ `npm run build` réussit (frontend)
 - ☐ `pytest` passe (backend)
 - ☐ Pas de `console.log()` oubliés
 - ☐ `.env` non commité
 - ☐ Commentaires ajoutés si logique complexe
 - ☐ Nommage variables explicite
 - ☐ Pas de code commenté inutile
-



PRÉSENTATION PFE

Points Clés à Mentionner

Architecture

- ☒ Microservices (découplage, scalabilité)
- ☒ Event-driven (Redis pub/sub)
- ☒ WebSocket temps réel
- ☒ Multi-tenant avec isolation

Technologies

- ☒ React + Vite (frontend moderne)
- ☒ Flask (microservices Python)
- ☒ TensorFlow/Keras (CNN pour ML)

- ☒ PostgreSQL (relationnel)
- ☒ Redis (cache + messaging)
- ☒ Docker (conteneurisation)
- ☒ MQTT (protocole IoT)

Fonctionnalités

- ☒ Détection défauts temps réel
- ☒ Dashboard avec KPIs
- ☒ Assignment alertes techniciens
- ☒ Export PDF rapports
- ☒ Heatmap alertes 24h
- ☒ Graphiques capteurs temps réel

Bonnes Pratiques

- ☒ Code modulaire et DRY
- ☒ Tests unitaires et intégration
- ☒ Documentation complète
- ☒ Git workflow (feature branches)
- ☒ Docker pour reproductibilité

Document généré le: 2026-08-20

Version: SmartMaintain V7

Auteur: Équipe SmartMaintain

Status: ☒ Production Ready